Practical : 2	Implementation and Time analysis of Linear Search and Binary Search algorithm.
Date : 12/08/2026	

Objective :- To implement Linear Search and Binary Search in C++, and to analyse and compare their time complexity (best, average, worst case) through practical execution-time measurements on inputs of varying size.

Program :-

```
#include <iostream>
#include <vector>
#include <chrono>

using namespace std;
using namespace std::chrono;

// ---------------- Linear Search ----------------
int linearSearch(const vector<int>& arr, int key)
{
   for (int i = 0; i < (int)arr.size(); i++)
   {
      if (arr[i] == key)
         return i;
   }

   return -1;
}

// ---------------- Binary Search ----------------
int binarySearch(const vector<int>& arr, int key)
{
```

```
    int low = 0;
    int high = (int)arr.size() - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

// Number of trials for timing
const int TRIALS = 5000;

// ---------------- Measure Average Time ----------------
template <typename Func>
double measureAvgTime(Func f, const vector<int>& arr, int key)
{
    volatile int sink = 0;

    auto start = high_resolution_clock::now();

    for (int t = 0; t < TRIALS; t++)
    {
        sink += f(arr, key);
    }
```

```
    auto stop = high_resolution_clock::now();

    (void)sink;

    return duration_cast<microseconds>(stop - start).count()
           / (double)TRIALS;
}

// ---------------- Main Function ----------------
int main()
{
    vector<int> sizes = {100, 1000, 5000, 10000, 50000, 100000};

    cout << "Size\tLinear(worst)\tBinary(worst)"
         << "\t(avg of " << TRIALS << " runs, microseconds)\n";

    for (int n : sizes)
    {
        vector<int> arr(n);

        // Sorted array of even numbers
        for (int i = 0; i < n; i++)
        {
            arr[i] = i * 2;
        }

        // Absent value -> worst case for both algorithms
        int key = -1;

        double tLinear = measureAvgTime(linearSearch, arr, key);
        double tBinary = measureAvgTime(binarySearch, arr, key);
```

```
        cout << n << "\t"
             << tLinear << "\t\t"
             << tBinary << "\n";
    }

    return 0;
}
```

Analysis:-

Time Complexity Comparison:

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Linear Search	Θ(1)	Θ(n)	O(n)	O(1)	Yes
Binary Search	Θ(1)	Θ(log n)	O(log n)	O(1)	Yes

Observed Execution Time on a sorted array of size n, searching for a key that is absent (worst case for both algorithms). Each figure is the average of 5000 repeated runs to obtain a stable, measurable reading:

Input Size (n)	Linear Search (worst case, avg of 5000 runs, μs)	Binary Search (worst case, avg of 5000 runs, μs)
100	0.073	0.0085
1000	0.626	0.0140
5000	3.151	0.0156
10000	6.162	0.0168
50000	31.525	0.0236
100000	62.575	0.0319

Linear Search shows clear linear growth in execution time as n increases, since in the worst case it must examine every element of the array before concluding the key is absent. Binary Search, by contrast, grows extremely slowly with n because it discards half of the remaining search space at every comparison, so its worst-case cost is proportional to $\log_2(n)$ rather than n. Between n = 100 and n = 100000 (a 1000x increase in input size), Linear Search's time increases by roughly the same 1000x factor, while Binary Search's time increases by less than 4x — closely matching the theoretical O(n)

versus O(log n) behaviour. The trade-off is that Binary Search requires the input array to be sorted beforehand, whereas Linear Search works on unsorted data and needs no pre-processing.

Conclusion :- Linear Search and Binary Search were successfully implemented and their execution times measured across increasing input sizes. Linear Search, while simple and applicable to unsorted data, runs in O(n) time and becomes impractical for large datasets when repeated lookups are required. Binary Search, requiring a sorted array as a precondition, achieves O(log n) time and scales dramatically better, confirming the theoretical complexity bounds. This demonstrates that when data can be pre-sorted (or is naturally maintained in sorted order) and searched repeatedly, the onetime cost of sorting is easily offset by the large efficiency gain of Binary Search over Linear Search.